KOENIG

step forward



Kotlin - Interoperability with Java & Spring Boot

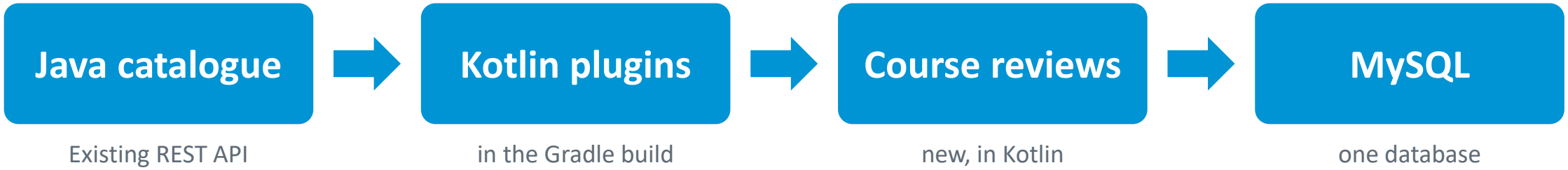
Kotlin for Java Developers

What You Will Learn

1. Kotlin in the Gradle project
2. Kotlin calls Java
3. Java calls Kotlin
4. DTOs, validation, Jackson
5. Spring Data JPA, and Exception Handling

Demo Scenario: Learnly Adds Reviews in Kotlin

The Java catalogue API works and stays. Learnly's next feature, course reviews, is written in Kotlin in the same project; then two Java classes are converted while the API keeps running.



What can go wrong

- **Final classes**
- **Nulls from Java**
- **Lost statics**

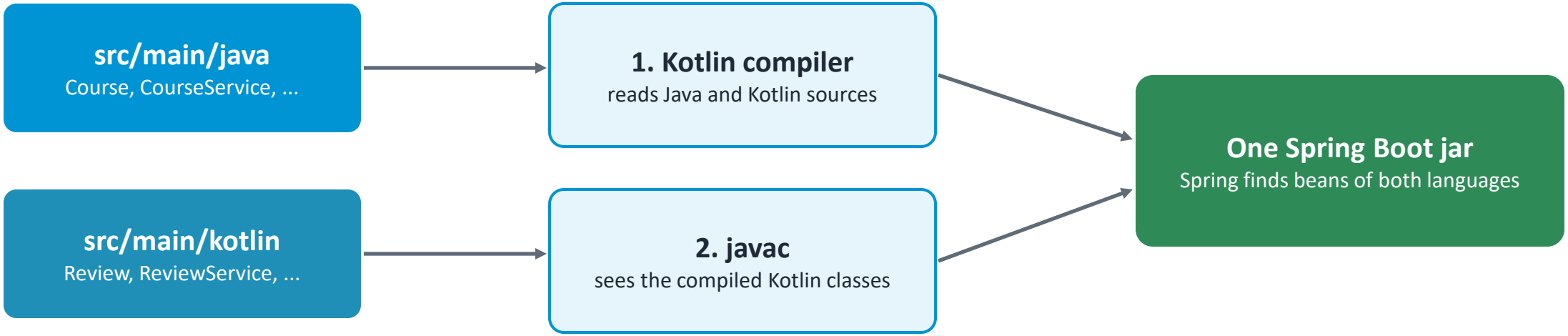
Why Kotlin Beside Java

	Keep as Java	Write in Kotlin
Tested, stable code	do not rewrite it	
New features		less code, null-safe types
Libraries and Spring	the same ones	the same ones
Team and delivery	no big-bang rewrite	one class at a time

Why it works: one JVM, one bytecode format, one jar.
What we learn: the places where the two languages meet.

No rewrite: new code in Kotlin, old code stays

Java and Kotlin in One Project



Both directions: Kotlin calls Java and Java calls Kotlin inside the same module.
Spring: does not care which language wrote a bean.

Two source folders, one build, one Spring Boot jar

Kotlin in the Gradle Build

```
plugins {
    java
    kotlin("jvm") version "2.4.10"
    kotlin("plugin.spring") version "2.4.10"
    kotlin("plugin.jpa") version "2.4.10"
    ...
}

dependencies {
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    implementation("tools.jackson.module:jackson-module-kotlin")
}
```

Entry	Job
jvm	compiles Kotlin
plugin.spring	opens Spring classes
plugin.jpa	no-arg for entities
kotlin-reflect	Spring reads Kotlin
jackson-module-kotlin	data classes in JSON

Click Sync: after every change in build.gradle.kts.

Three plugins and two libraries

Mixed Sources and the Compile Order

Folder	Contains	Compiled by
src/main/java	Course, CourseService	javac, second
src/main/kotlin	reviews, AboutController	kotlinc, first
src/main/resources	application.properties	copied

Same packages: controller, service, repository, entity, dto, exception in both folders.
Names: a Java caller sees what the Kotlin compiler generated: file classes, companions, getters.

Where the files go and what compiles first

The spring and jpa Plugins

Plugin	Effect on the class	Needed for
plugin.spring (all-open)	classes with @Component, @Service, @Transactional, @Cacheable ... become open	proxies for caching, transactions
plugin.jpa (no-arg)	@Entity classes get a no-argument constructor	Hibernate loads rows

Without spring: a final class with @Cacheable cannot be subclassed: the application does not start.
Without jpa: saving works, loading a row fails: No default constructor for entity.

Kotlin classes are final: two plugins undo that for the frameworks

Demo: Kotlin in the Gradle Project

What we do

- **Run:** the Java API as it is.
- **Build:** plugins and libraries.
- **Write:** the first Kotlin controller.

What it shows

- **Same answers:** the Java API is unchanged.
- **One build:** Kotlin and Java together.
- **Spring:** finds the Kotlin controller.

Records, Optional and Lambdas from Kotlin

```
val course = courses.get(id)                        // Java CourseService
return "${course.title} (${course.status}, launches ${course.launchDate})"

val course: Course? = repository.findById(id).orElse(null)
return course?.category ?: "unknown course"
```

Java	In Kotlin
record component	property
Optional	orElse(null)
RowMapper	lambda
checked exception	not declared

The Java API starts to feel like Kotlin

File Classes and @file:JvmName

```
// Slugs.kt
package com.example.catalogdb.support

fun slugify(title: String): String =
    title.lowercase().replace(Regex("[^a-z0-9]+"), "-
").trim('-')
```

```
// Java
SlugsKt.slugify(title)

@file:JvmName("Slugs")      // first line of Slugs.kt

Slugs.slugify(title)
```

Default: file name + Kt, every top-level function is static.
@file:JvmName: your own class name; change the Java call sites too.

What a Java caller sees of a top-level Kotlin function

Demo: Kotlin Calls Java & Java Calls Kotlin



What we do

- **Call:** the Java service from Kotlin.
- **Break:** a nullable from Spring.
- **Smooth:** Optional and a lambda.
- **Call:** SlugsKt.slugify from Java.

What it shows

- **Properties:** record components.
- **Same handler:** Java exception, 404.
- **File class:** Kt suffix by default.
- **Name:** @file:JvmName.

Data Class DTOs

```
data class ReviewRequest(
    val rating: Int,
    val comment: String,
    val anonymous: Boolean = false
)

data class ReviewResponse(
    val id: Long?, val courseId: Long, val rating: Int,
    val comment: String, val reviewer: String
)
```

Kotlin	Meaning in JSON
val comment: String	required, never null
val note: String?	optional, may be null
val anonymous = false	optional, default

Same JSON as a Java record: the property names are the field names.

Kotlin data class is the natural DTO

Validation Annotations on Data Classes

```
data class ReviewRequest(
    @Min(1) @Max(5) val rating: Int,
    @NotBlank @Size(max = 500) val comment: String,
    val anonymous: Boolean = false
)

fun create(@PathVariable courseId: Long,
           @Valid @RequestBody request: ReviewRequest):
ReviewResponse
```

Written as	Lands on
@NotBlank val x	where the validator looks (Kotlin 2.4)
@field:NotBlank val x	the field, explicit
@get:NotBlank val x	the getter

The same handler: the Java ApiExceptionHandler returns the 400 problem detail for Kotlin DTOs too.

Bean validation works on constructor properties

The Jackson Kotlin Module

Request body	Result
{"rating": 5, "comment": "Clear and practical"}	201; anonymous = false (the default)
{"rating": 4, "comment": "Good pace", "anonymous": true}	201; reviewer is anonymous
{"rating": 4}	400: comment is missing and not nullable
{"rating": 9, "comment": ""}	400 from validation (after parsing)

Two layers: Jackson checks the shape, bean validation checks the values.

Reads Kotlin's types, defaults and nullability

Demo: DTOs, Validation and Jackson

What we do

- **Echo:** a Kotlin controller and data classes.
- **Validate:** @Min, @Max, @NotBlank, @Valid.
- **Parse:** missing and default properties.

What it shows

- **No boilerplate:** a data class per DTO.
- **Same 400:** the Java handler answers.
- **Types:** Jackson knows Kotlin.

Entities in Kotlin

```
@Entity
@Table(name = "reviews")
class Review(
    @Column(nullable = false)
    var courseId: Long,

    @Column(nullable = false)
    var rating: Int,

    @Column(name = "comment_text", nullable = false, length = 500)
    var comment: String,

    @Column(nullable = false, length = 60)
    var reviewer: String
) {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Long? = null
}
```

var: Hibernate sets the values.
Long? = null: no id before save.
Annotations: on the properties.
plugin.jpa: the no-arg constructor.

A normal class with var properties and a nullable id

Demo: Kotlin Entities

What we do

- **Map:** Review as a Kotlin class.
- **Persist:** repository, service, controller.
- **Break:** no jpa plugin; a data class.

What it shows

- **Table:** created from the annotations.
- **Load:** needs the no-arg constructor.
- **Identity:** a data class loses its hash.

Repositories in Kotlin

```
interface ReviewRepository : JpaRepository<Review, Long> {
    fun findByCourseId(courseId: Long): List<Review>

    fun countByCourseId(courseId: Long): Long
}

import org.springframework.data.repository.findByIdOrNull

fun get(id: Long): ReviewResponse =
    reviews.findByIdOrNull(id)?.toResponse() ?: throw
ReviewNotFoundException(id)
```

Java	Kotlin
Optional<T>	T? with findByIdOrNull
List<T>	List<T>
orElseThrow	?: throw

A Kotlin interface, derived queries and findByIdOrNull

Kotlin Collections and Extension Mappers

```
val ratings = reviews.findByCourseId(courseId).map { it.rating }
return ReviewSummary(
    courseId = courseId,
    count = ratings.size,
    average = if (ratings.isEmpty()) 0.0 else round(ratings.average() * 10) / 10,
    histogram = (1..5).associateWith { star -> ratings.count { it == star } }
)

fun Review.toResponse() = ReviewResponse(id, courseId, rating, comment, reviewer)

reviews.findByCourseId(courseId).map(Review::toResponse)
```

map, average, count: Module 17.
Extension mapper: Module 16.
Data class: Module 15.

Build the summary and the DTOs with what Module 16 and 17 taught

Demo: Spring Data in Kotlin



What we do

- **Find:** findByIdOrNull.
- **Summarise:** collections build the histogram.
- **Refactor:** toResponse as an extension.

What it shows

- **No Optional:** ?. and ?: instead.
- **Few lines:** average, count, associateWith.
- **Same answers:** after the refactoring.

Problem Details in Kotlin

```
class ReviewLimitExceededException(val limit: Int) :
    RuntimeException("This course already has $limit reviews")

@RestControllerAdvice
class ReviewErrors {

    @ExceptionHandler(ReviewLimitExceededException::class)
    fun limitExceeded(ex: ReviewLimitExceededException): ProblemDetail =
        ProblemDetail.forStatusAndDetail(
            HttpStatus.CONFLICT, ex.message ?: "Too many reviews"
        ).apply {
            title = "Review limit reached"
            setProperty("limit", ex.limit)
        }
}
```

apply { }: configure the ProblemDetail.
ex.message ?: ...: message is nullable in Kotlin.
setProperty: extra JSON fields.

A @RestControllerAdvice written in Kotlin

Two Advices, One Application

Advice (language)	Handles	Status
ApiExceptionHandler (Java)	validation, CourseNotFound, PaymentFailed, sort	400, 404, 402
ReviewErrors (Kotlin)	ReviewNotFound, ReviewLimitExceeded	404, 409

One handler per exception: no overlap, no surprises.
Overlap: @Order(Ordered.HIGHEST_PRECEDENCE) decides which advice wins.
Base class: only one advice extends ResponseEntityExceptionHandler.

Java and Kotlin handlers side by side

Demo: Errors in Kotlin

What we do

- **Handle:** an unknown review in Kotlin.
- **Rule:** a review limit and a 409.
- **Order:** which advice wins.

What it shows

- **Side by side:** Java and Kotlin advices.
- **Extra field:** limit in the problem detail.
- **Overlap:** @Order decides.

Configuration Properties as a Data Class

```
@ConfigurationProperties("learnly.reviews")
data class ReviewProperties(val maxPerCourse: Int = 3)

learnly.reviews.max-per-course=2
```

```
@SpringBootApplication
@EnableCaching
@ConfigurationPropertiesScan      // the Java application class
public class CatalogDbApp { ... }
```

Case	Result
no property	default 3
max-per-course=2	limit 2
max-per-course=many	start-up failure
@Value("\${...}")	backslash before $

Typed settings with defaults, bound through the constructor

Demo: Configuration Properties

What we do

- **Bind:** a data class with a default.
- **Set:** application.properties.
- **Break:** a value that is not a number.

What it shows

- **Java app class:** @ConfigurationPropertiesScan.
- **Same rule:** the limit comes from the setting.
- **Fail fast:** the property is named.

Key Takeaways

1. **One project**
   Kotlin and Java build together; three plugins and two libraries.

2. **Kotlin calls Java**
   Platform types, Optional, lambdas, same exceptions.

3. **Spring in Kotlin**
   Constructor injection; plugin.spring opens the proxied classes.

4. **Data and JPA**
   Data class DTOs, normal class entities, findByIdOrNull, collections.

5. **Errors and migration**
   Kotlin advices, data class properties, one class at a time.



Happy Learning :)